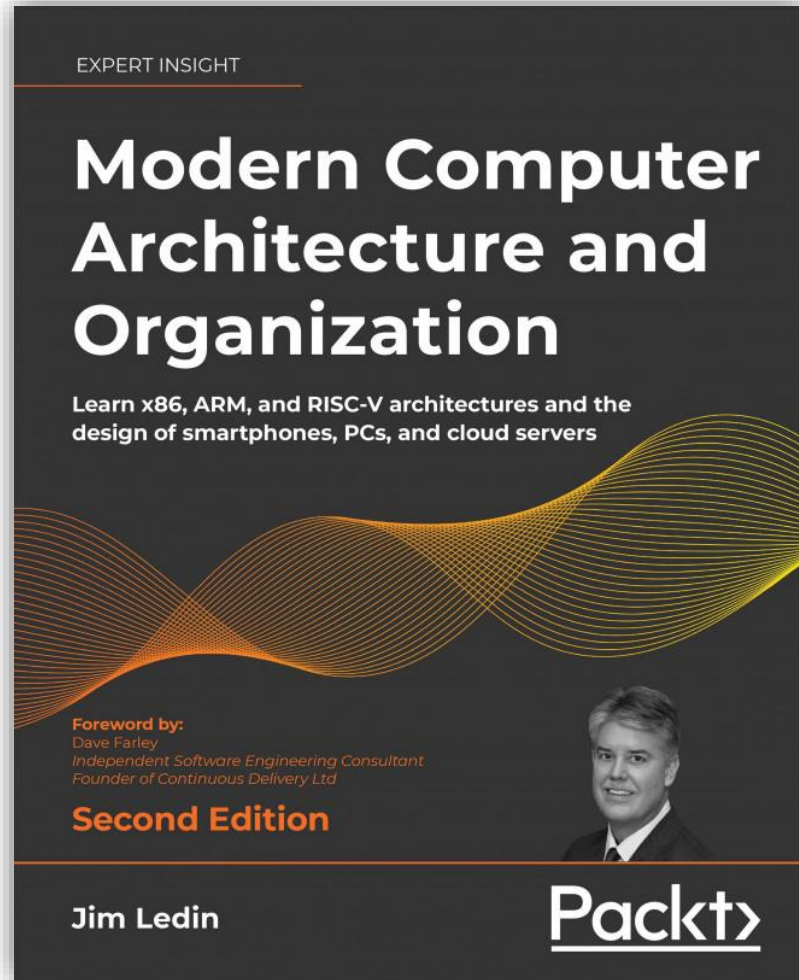


Computer Architecture

Chapter 6 – Specialized Computing Domains



VIVES University of Applied Sciences Bachelor in electronics-ICT



— Computer Architecture

Chapter 6: Specialized Computing Domains

Contents

1. **Real-time computing**
2. GPU processing
3. Examples of specialized architectures

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing

- Real-time computing systems can be categorized as providing **soft** or **hard** guarantees of *responsiveness*.
- A **soft real-time system** is considered to perform acceptably if it meets its desired response time most, but not necessarily all of the time.
- An example of a **soft** real-time application is the **clock display on a cell phone**. When opening the clock display, some implementations momentarily present the time that was shown the last time the clock display was opened before quickly updating to the correct, current time.
- Of course, users would like the clock to show the correct time whenever it is displayed, but momentary glitches such as this aren't usually seen as a significant problem.
- A **hard real-time system**, on the other hand, is considered to have failed if it ever misses any of its response-time deadlines. Safety-critical systems such as **airbag controllers** in **automobiles** and **flight control systems** for **commercial aircraft** have hard real-time requirements.
- Designers of these systems take timing requirements very seriously and devote substantial effort to ensuring the real-time processor satisfies its timing requirements under all possible operating conditions.

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Types of real Time Systems

Types of Real Time Systems

- **Hard real time systems**
 - Must always meet all deadlines
 - System fails if deadline window is missed
- **Soft real time systems**
 - Must try to meet all deadlines
 - System does not fail if a few deadlines are missed
- **Firm real time systems**
 - Result has no use outside deadline window
 - Tasks that fail are discarded

Figure 6.1: Types of Real Time Systems

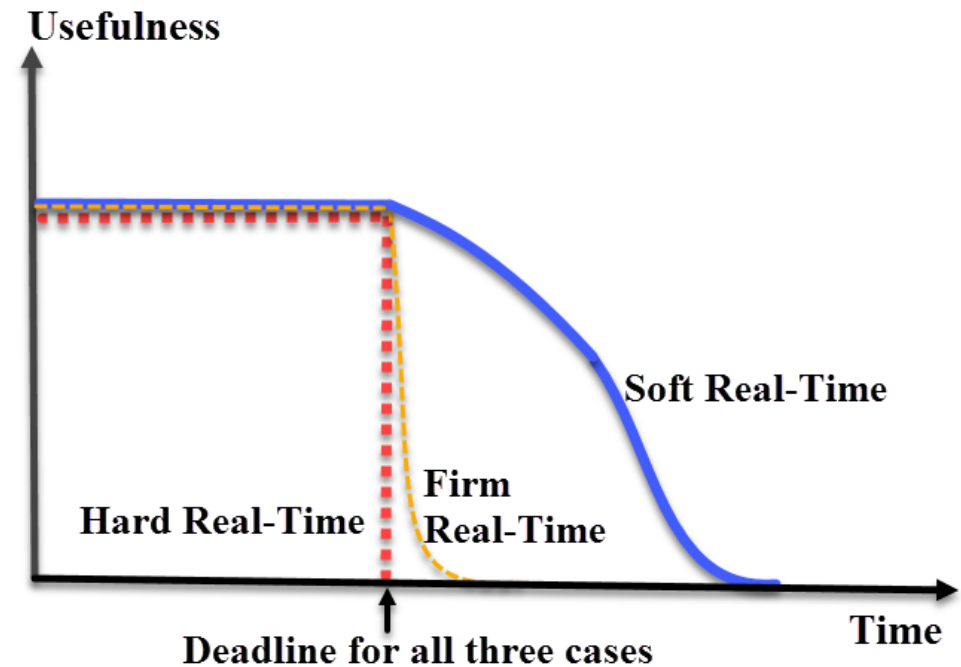


Figure 6.2: Deadlines in Real Time Systems

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing

- The control flow of a simple real-time system is shown in **Figure 6.3**.
- **Figure 6.3** represents a real-time computing system using a **hardware interval timer** to control the time sequencing of its operation.
- A down-counting interval timer performs a repetitive cycle of the following steps:
 1. Load the counter register with a predefined numeric value.
 2. Decrement the counter at a fixed clock rate.
 3. When the count reaches **zero**, generate an event such as setting a bit in a register or triggering an **interrupt**.
 4. Go back to Step 1.
- An **interval timer** generates a periodic sequence of events with timing accuracy that depends on the accuracy of the system clock, which is often driven by a quartz crystal.
- By waiting for the timer event at the top of each loop, the system in **Figure 6.3** begins each execution pass at fixed, equal time intervals.

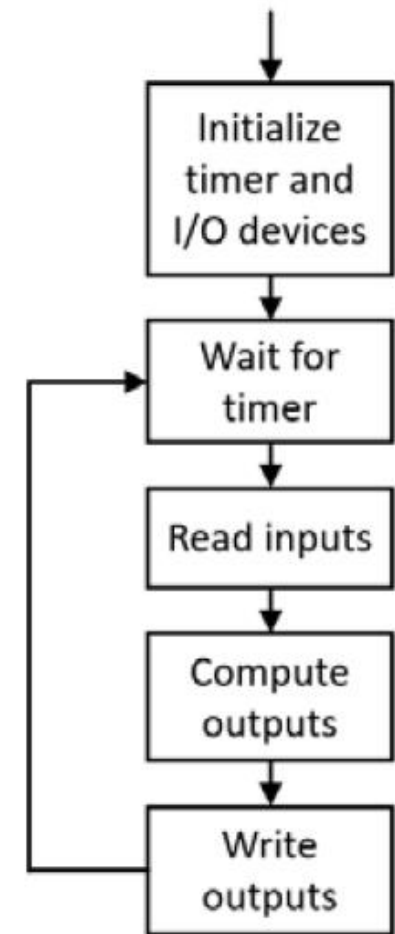


Figure 6.3: Real-time system control flow

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing

- To satisfy the demands of **hard real-time** operation, *the execution time of the code inside the loop* (the code contained in the **Read inputs**, **Compute outputs**, and **Write outputs** blocks in **Figure 6.3**) *must always be less than the timer interval*.
- Prudent system developers ensure that no path through the code results in execution time anywhere close to the hard real-time limit. A conservative system design rule might insist the longest execution path for code inside the loop consumes no more than 50% of the timer interval.
- Practical real-time systems constructed in this configuration might be based on an 8-, 16-, 32-bit or 64-bit processor running at a clock frequency in the tens to hundreds of MHz.
- The timer employed in the main loop of such systems generates events at a developer-selected frequency, often in the range 10 to 1000 Hz.
- The code represented in **Figure 6.3** runs directly on the processor hardware with no intervening software layers. This configuration contains *no operating system* of the type described in **Chapter 5, Hardware-Software Interface**.
- A sophisticated real-time application is likely to have more extensive needs than can be met by this simplistic architecture, making the use of a **real-time operating system (RTOS)** attractive.

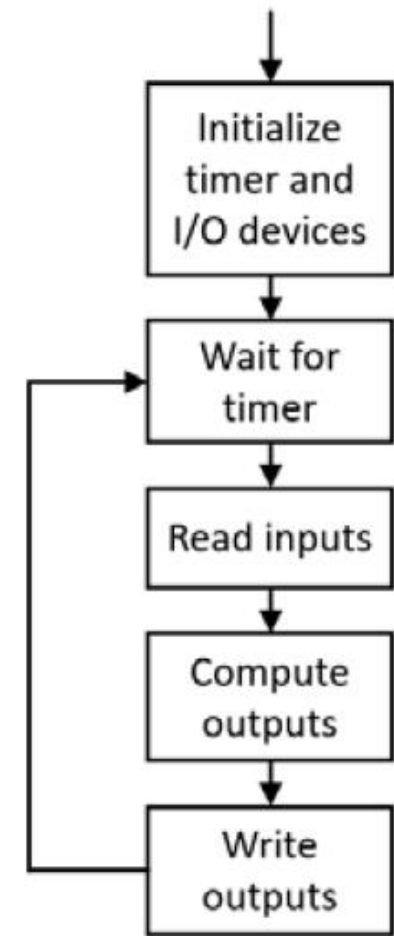


Figure 6.3: Real-time system control flow

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- A **real-time operating system (RTOS)** contains several features superficially similar to the general-purpose operating systems discussed in *Chapter 5, Hardware-Software Interface*.
- An **RTOS** design differs significantly from general-purpose operating systems however, in that all RTOS aspects—from *kernel internals*, to *device drivers*, to *system services*— are focused on **meeting hard real-time requirements**.
- Most RTOS designs employ **preemptive multithreading**, often referred to as **multitasking** in RTOS literature.
- The terms **task** and **thread** are somewhat synonymous in the RTOS context, so for consistency we will continue to use the term **thread** to indicate an **RTOS task**.
- RTOS designs at the lower end of the sophistication scale typically support multithreading within the context of a **single application process**. These simpler RTOSes support **thread prioritization** but often **lack memory protection features**.
- More sophisticated RTOS architectures provide **multiple thread privilege levels** and **operating system features** such as **memory protection**, in addition to **prioritized, preemptive multithreading**.
- These RTOSes allow multiple processes to be in the *Running* state simultaneously, each potentially containing several threads. In **protected memory systems**, kernel memory access by application threads is prohibited and applications cannot reach into each other's memory regions.
- Many RTOSes support **multi-core processor architectures**.

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: RTOS versus GPOS

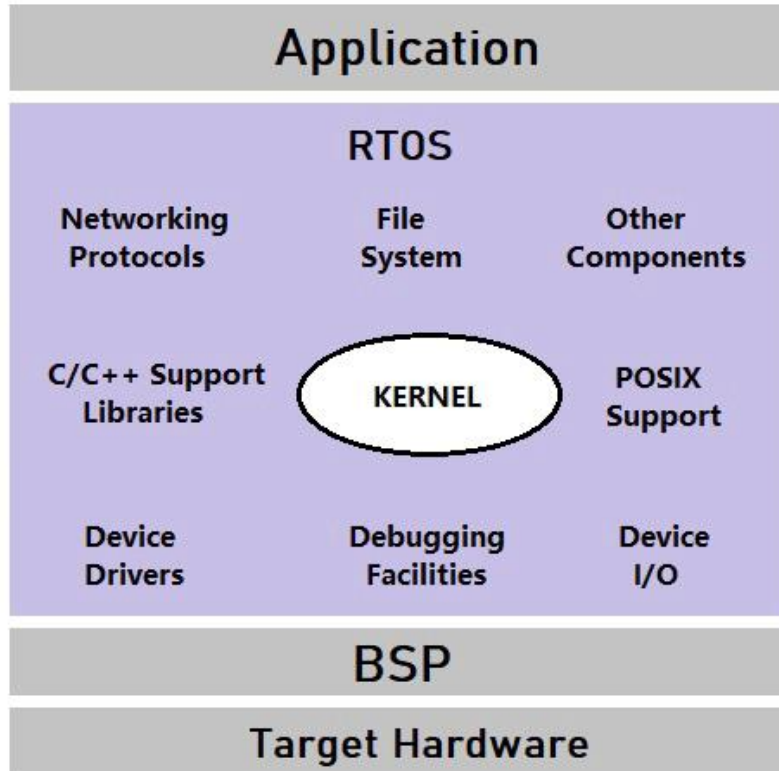


Figure 6.4: Structure of RTOS



RTOS and GPOS Differences

RTOS

Real-Time Operating System

- Deterministic: no random execution pattern
- Predictable Response Times
- Time Bound
- Preemptive Kernel

Examples:

Contiki source code, FreeRTOS™, Zephyr™ Project

Use Case:

Embedded Computing

GPOS

General-Purpose Operating System

- Dynamic memory mapping
- Random Execution Pattern
- Response Times not Guaranteed

Examples:

Microsoft® Windows® operating system, Apple® macOS® operating system, Red Hat® Enterprise Linux® operating system

Use Case:

Desktop, Laptop, Tablet computers

Figure 6.5: RTOS and GPOS differences

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- RTOS environments, from lower to higher levels of sophistication, provide several data structures and communication techniques geared toward efficient data transfer between threads, and to support controlled access to shared resources.
- Some examples of these features are as follows:
 - **Mutex:** A **mutex** (short for *mutual exclusion*) is a mechanism for a thread to claim access to a **shared resource**, without blocking the execution of other threads. In its simplest form, a mutex is a **variable** accessible to *all threads* that has the value **0** when the resource is **free** and **1** when the resource is **in use**. A thread that wants to use the resource reads the current value of the mutex variable and, if it is 0, sets it to 1 and performs the operation with the resource. After completing the operation, the thread sets the mutex back to 0. There are, however, some potential problems with mutexes:
 - **Thread preemption:** Let's say a thread reads the mutex variable and sees that it is **0**. Because the scheduler can interrupt an executing thread at any time, *that thread might be interrupted before it has a chance to set the mutex to 1*. A different thread then resumes execution and takes control of the same resource because it sees the mutex is still **0**. When the original thread resumes, it finishes setting the mutex to **1** (even though, by now, it has already been set to 1). *At this point, both threads incorrectly believe they have exclusive access to the resource, which is likely to lead to serious problems.*

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: MUTEX

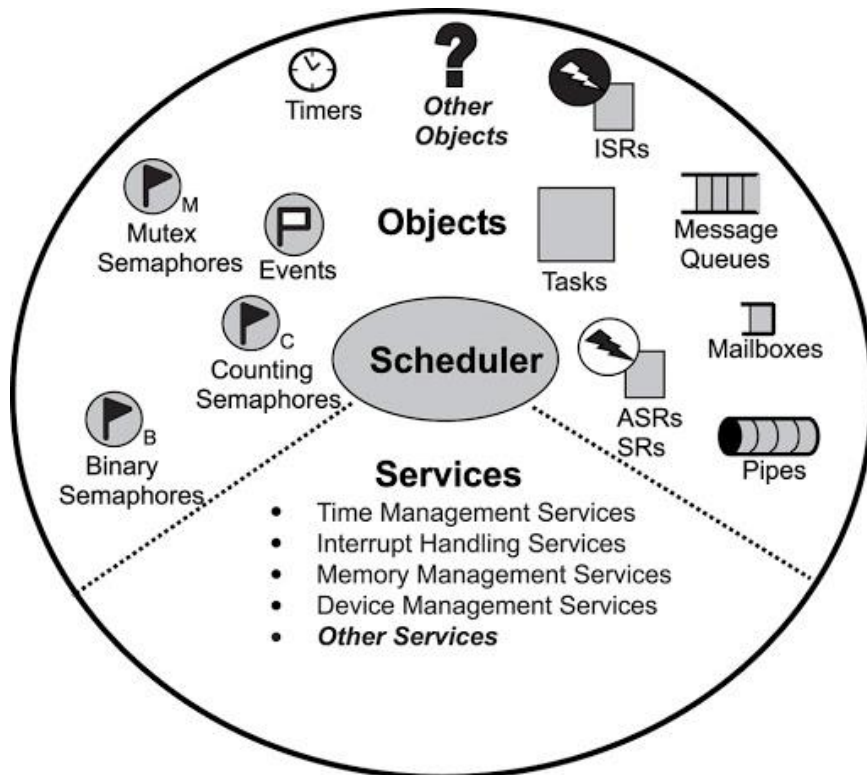


Figure 6.6: OS Scheduler Objects

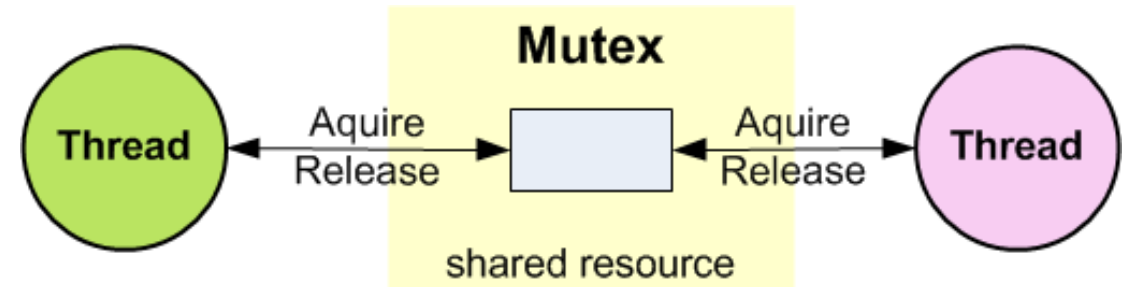


Figure 6.7: MUTEX management of shared resource

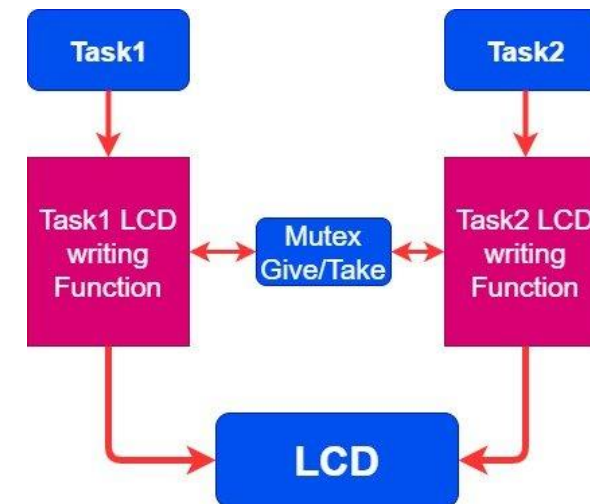


Figure 6.8: MUTEX management of shared resource

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- Some examples of these features are as follows:
 - **Mutex:**
 - There are, however, some potential problems with mutexes:
 - **Thread preemption:**
 - To prevent this scenario, many processors implement some form of *a test-and-set instruction*.
 - A *test-and-set instruction* reads a value from a memory address and sets that location to **1** in a **single, uninterruptable** (also referred to as **atomic**) **action**.
 - In the x86 architecture, the **BTS** (bit test and set) instruction performs this atomic operation. In processor architectures that lack a *test-and-set instruction* (such as the 6502), the risk of preemption can be eliminated by **disabling interrupts before checking the state of the mutex variable**, then re-enabling interrupts after setting the mutex to **1**.
 - This approach has the disadvantage of reducing real-time responsiveness while interrupts are disabled.

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- Some examples of these features are as follows:
 - **Mutex:**
 - There are, however, some potential problems with mutexes:
 - **Priority inversion:**
 - **Priority inversion** occurs when a *higher-priority* thread attempts to gain access to a resource while the corresponding mutex is held by a *lower-priority* thread. In this situation, RTOS implementations generally place the **higher-priority** thread in a **blocked** state, allowing the **lower-priority** thread to complete its operation and release the mutex.
 - The **priority inversion problem** occurs when a thread with a priority *between the upper and lower levels* of the two threads begins execution. *While this mid-priority thread is running, it prevents the lower-priority thread from executing and releasing the mutex.*

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- Some examples of these features are as follows:
 - **Mutex:**
 - There are, however, some potential problems with mutexes:
 - **Priority inversion:**
 - The **higher-priority thread** must now **wait** until the **mid-priority thread** finishes execution, effectively disrupting the entire thread prioritization scheme. *This can lead to failure of the high-priority thread to meet its deadline.*
 - One method to prevent priority inversion is **priority inheritance**.
 - In an RTOS implementing **priority inheritance**, whenever a *higher-priority thread* (thread2) requests a mutex held by a *lower-priority thread* (thread1), thread1 is temporarily raised in priority to the level of thread2.
 - *This eliminates any possibility of a mid-priority thread delaying the completion of the (originally) lower-priority thread1.*
 - When thread1 releases the mutex, the RTOS restores its original priority.

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- Some examples of these features are as follows:
 - **Deadlock:**
 - **Deadlock** can occur when multiple threads attempt to lock multiple mutexes.
 - If *thread1* and *thread2* both require *mutex1* and *mutex2*, a situation may arise in which *thread1* locks *mutex1* and attempts to lock *mutex2*, while at the same time *thread2* has already locked *mutex2* and attempts to lock *mutex1*.
 - Neither task can proceed from this state, hence the term **deadlock**.
 - Some RTOS implementations check the ownership of mutexes during lock attempts and report an error in a deadlock situation.
 - *In simpler RTOS designs, it is up to the system developer to ensure deadlock cannot occur.*

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- Semaphore:

- A **semaphore** is a generalization of the **mutex**.
- Semaphores can be of two types: **binary** and **counting**.
- A **binary semaphore** is similar to a **mutex** except that rather than controlling access to a resource, the binary semaphore is intended to be used by one task to send a signal to another task. *If thread1 attempts to take semaphore1 while it is unavailable, thread1 will block until another thread or interrupt service routine gives semaphore1.*
- A **counting** semaphore contains a *counter* with an *upper limit*. Counting semaphores are used to control access to multiple interchangeable resources. When a thread takes the counting semaphore, the counter **increments** and the task proceeds. *When the counter reaches its limit, a thread attempting to take the semaphore blocks until another thread gives the semaphore, decrementing the counter.*
- Consider the **example** of a system that supports a limited number of simultaneously open files. A counting semaphore can be used to manage file open and close operations.
- If the system supports up to 10 open files and a thread attempts to open an 11th file, a counting semaphore with a limit of 10 will block the file open operation until another file is closed and its descriptor becomes available.

Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Semaphores

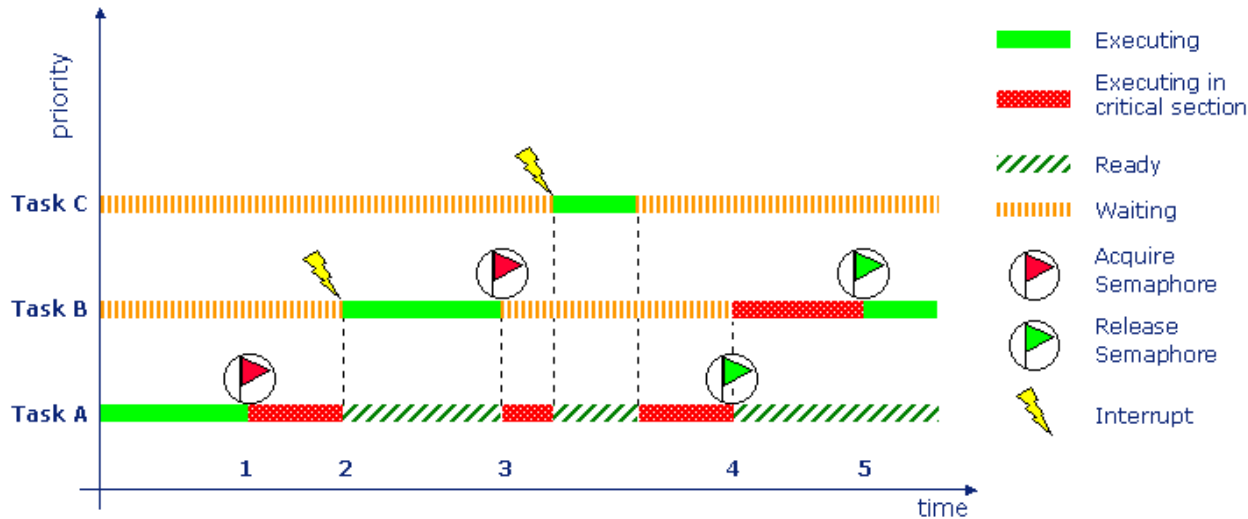


Figure 4. Mutual exclusion with a binary semaphore

1. Task A is executing and wants to access the resource. To be able to do this Task A must first acquire the semaphore.
2. Task B is made ready and preempts Task A still accessing the resource.
3. Task B wants to acquire the semaphore, but as it is busy, Task B will start to wait. Task A continues to execute.
4. Task A releases the semaphore. Task A is preempted by Task B, and Task B will now acquire the semaphore and get access to the shared resource.
5. Task B will then release the semaphore when it has finished using the shared resource.

Figure 6.9: binary semaphore

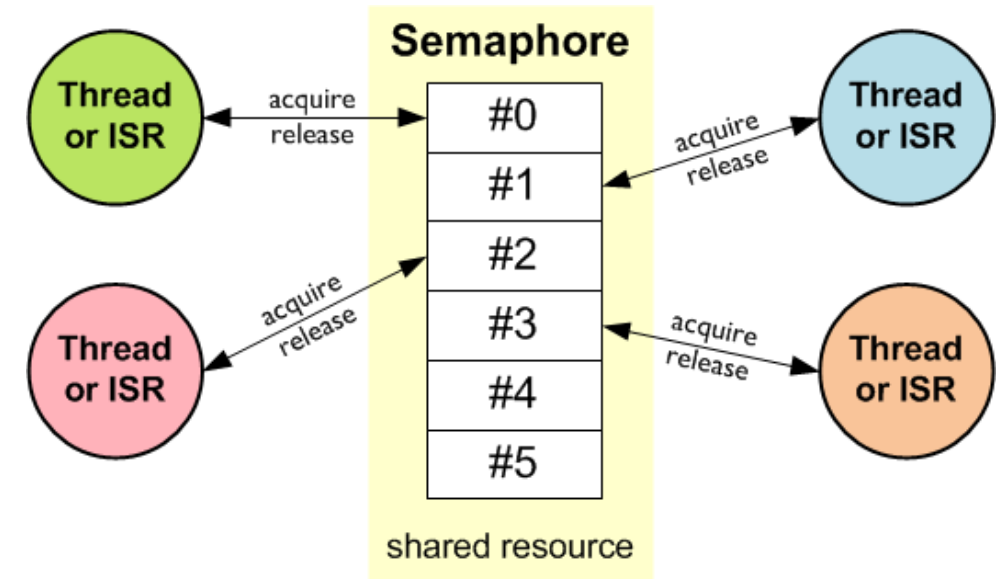


Figure 6.10: Semaphore management of shared resource

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- **Queue:**

- A **queue** (also referred to as a **message queue**) is a **unidirectional communication path** between *processes* or *threads*.
- The sending thread places data items into the queue and the receiving thread retrieves those items in the *same order* they were sent.
- The RTOS synchronizes accesses between the sender and receiver so the receiver only retrieves complete data items.
- **Queues** are commonly implemented with a *fixed-size storage buffer*. The buffer will eventually fill and block further insertions if a sending thread adds data items faster than the receiving thread retrieves them.
- RTOS message queues provide a programming interface for the receiving thread to check if the queue contains data.
- Many queue implementations also support the use of a semaphore to signal a blocked receiving thread when data is available.

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: message queues

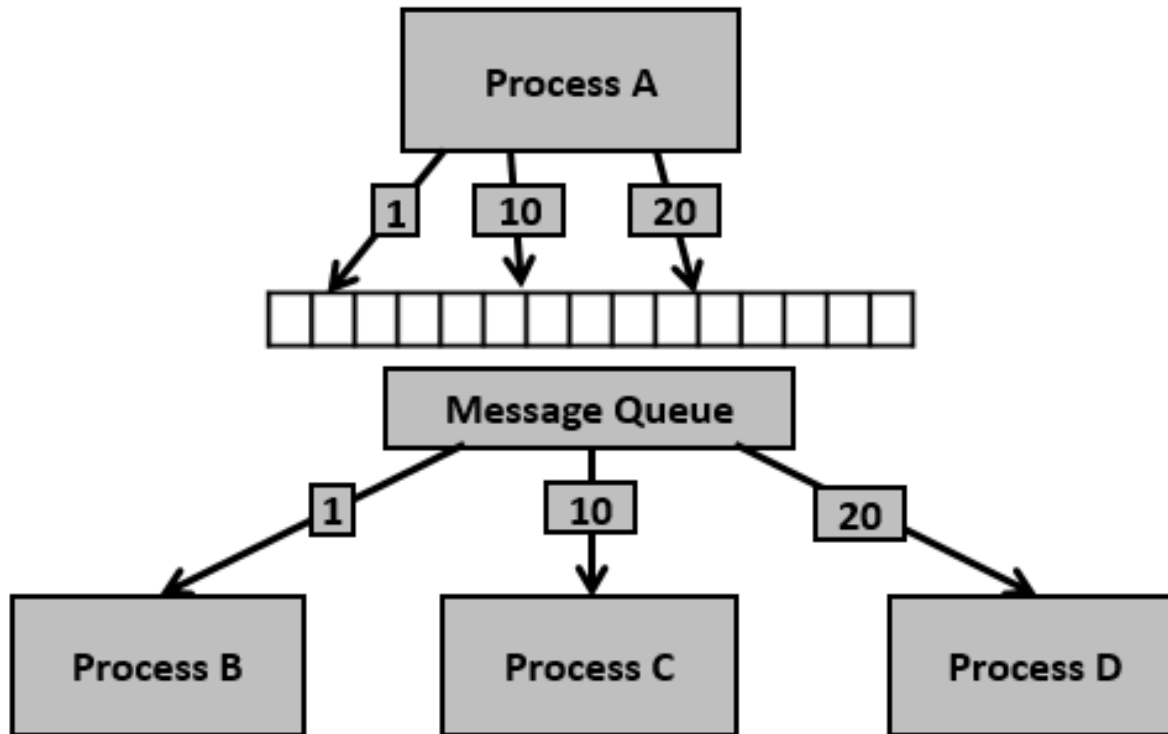


Figure 6.11: message queue

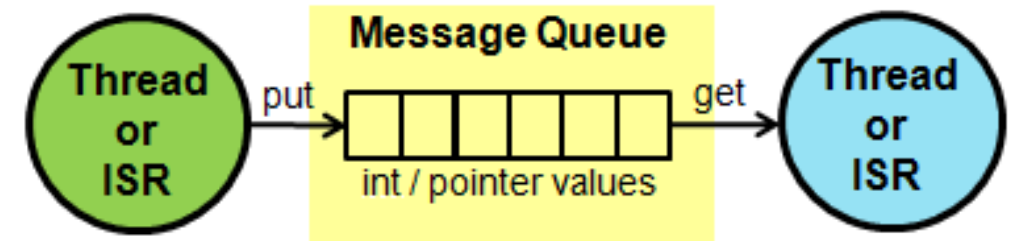


Figure 6.12: message queue

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- **Critical section:**
 - It is common for multiple threads to require access to a **shared** data structure.
 - When using **shared** data, it is vital that read and write operations from different threads *do not overlap in time*. If such an overlap occurs, the reading thread may receive inconsistent information if it accesses the data structure while another thread is in the midst of an update.
 - The **mutex** and **semaphore** mechanisms provide options for controlling access to such data structures. The use of a **critical section** is an alternate approach that isolates the code accessing the shared data structure and allows only one thread to execute that sequence at a time.

— Computer Architecture

Chapter 6: Specialized Computing Domains

1. Real-time computing: Real-time operating systems

- **Critical section:**
 - A simple method to implement a critical section is to **disable interrupts** just before entering a critical section and re-enable interrupts after completing the critical section.
 - This prevents the scheduler from running and ensures the thread accessing the data structure has sole control until it exits the critical section.
 - This method has the *disadvantage* of impairing real-time responsiveness by preventing responses to interrupts, including thread scheduling, while interrupts are disabled.
- Some RTOS implementations provide a more sophisticated implementation of the critical section technique, involving the use of **critical section data objects**.
- **Critical section objects** typically provide options to allow a thread to either enter a **blocked state** until the critical section becomes available or test if the critical section is in use without blocking.
- The option for testing critical section availability allows the thread to perform other work while waiting for the critical section to become free.

— Computer Architecture

Chapter 6: Specialized Computing Domains

Contents

1. Real-time computing
- 2. GPU processing**
3. Examples of specialized architectures

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing

- A **GPU** is a digital processor optimized to perform the mathematical operations associated with generating and rendering graphical images for display on a computer screen.
- The primary applications for GPUs are **playing video recordings** and **creating synthetic images of three-dimensional scenes**.
- The performance of a GPU is measured in terms of **screen resolution** (the pixel *width* and *height* of the image) and the **image update rate in frames per second**.
- **Video playback** and **scene generation** are **hard real-time** processes, in which any deviation from smooth, regularly time-spaced image updates is likely to be perceived by a user as unacceptable graphical stuttering.
- As with the video cameras, GPUs generally represent each pixel as three **8-bit color values**, indicating the intensities of **red**, **green**, and **blue**.
- Any color can be produced by combining appropriate values for each of these three colors. Within each color channel, the **value 0** indicates the color is **absent**, and **255** is **maximum intensity**. **Black** is represented by the color triple (red, green, blue) = (0, 0, 0), and **white** is (255, 255, 255).
- With 24 bits of color data, over 16 million unique colors can be displayed. The granularity between adjacent 24-bit color values is, in general, finer than the human eye can distinguish.

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing

- In modern personal computers, GPU functionality is available in a variety of configurations:
 - A GPU card can be installed in a **PCI Express (PCIe)** slot.
 - A system can provide a GPU as one or more discrete integrated circuits on the main processor board.
 - GPU functionality can be built into the main processor's integrated circuit.
- The **most powerful consumer-class GPUs** are implemented as **PCIe expansion cards**. These high-end GPUs contain dedicated graphics memory and feature a fast communication path with the main system processor (typically **PCIe x16**) for receiving commands and data representing the scene to be displayed. Some GPU designs support **the use of multiple identical cards in a single system** to generate scenes for a single graphical display.
- **This technology features a separate high-speed communication bus linking the GPUs to each other.** The use of multiple GPUs in a system effectively increases the **parallelization** of graphics processing.
- GPUs exploit the concept of **data parallelism** to perform identical computations simultaneously on a vector of data items, producing a corresponding vector of outputs.
- Modern GPUs support **thousands of simultaneously executing threads**, providing the capability to render complex, three-dimensional images containing millions of pixels at 60 or more frames per second.
- The architecture of a typical GPU consists of one or more multi-core processors, each supporting **multithreaded** execution of **data parallel algorithms**.

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing

- The **interface** between the **GPU processors** and **graphics memory** is optimized to provide **maximum data throughput**, rather than attempting to minimize access latency (which is the design goal for main system memory).
- GPUs can afford to sacrifice a degree of latency performance to achieve peak streaming rate between the GPU and its dedicated memory because maximizing throughput results in the highest possible frame update rate.
- In computer systems with **less extreme** graphical performance **demands**, such as **business applications**, a lower-performance GPU integrated on the same circuit die as the main processor is a lower-cost and often perfectly acceptable configuration.
- **Integrated GPUs** are able to play **streaming video** and provide a more limited level of three-dimensional scene-rendering capability compared to the higher-end GPUs.

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing

- Rather than using dedicated graphics memory, these integrated GPUs use **a portion of system memory** for graphics rendering. Although the use of system memory rather than specialized graphics memory results in a performance hit, such systems provide sufficient graphics performance for most **home and office purposes**.
- **Smart devices** such as portable phones and tablets contain GPUs as well, providing the same video playback and three-dimensional scene-rendering capabilities as larger non-portable computer systems. The constraints of **small physical size** and **reduced power consumption** necessarily limit the performance of portable device GPUs.
- Nevertheless, modern smartphones and tablets are fully capable of playing high-definition streaming video and rendering sophisticated gaming graphics.

— Computer Architecture

Chapter 6: Specialized Computing Domains

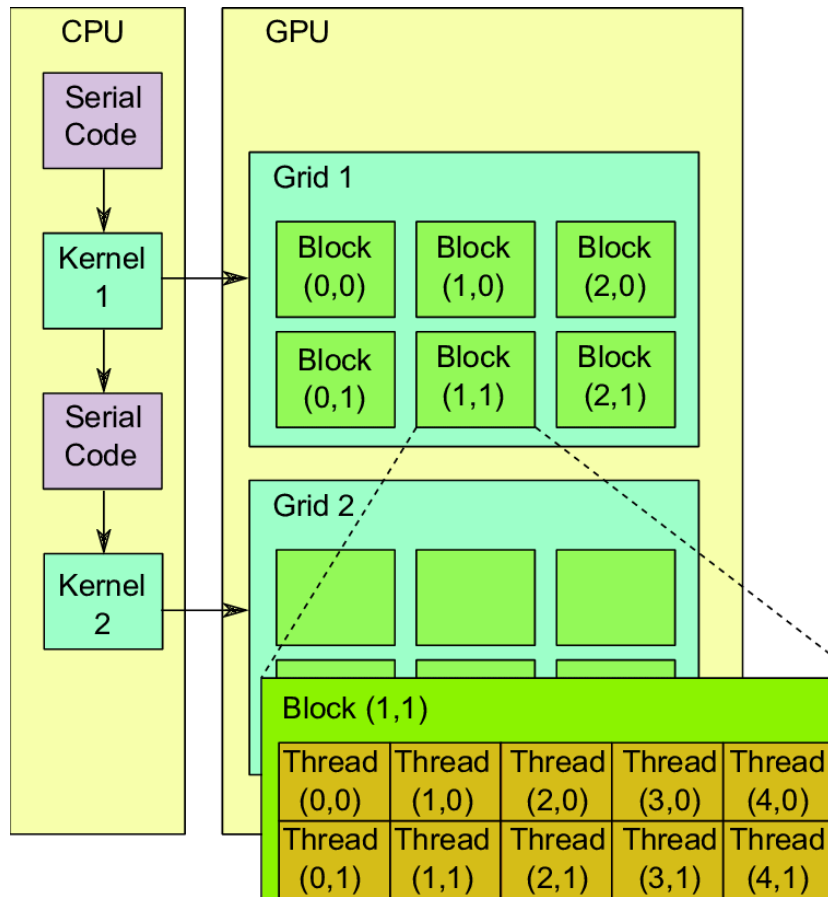


Figure 6.13: GPU architecture

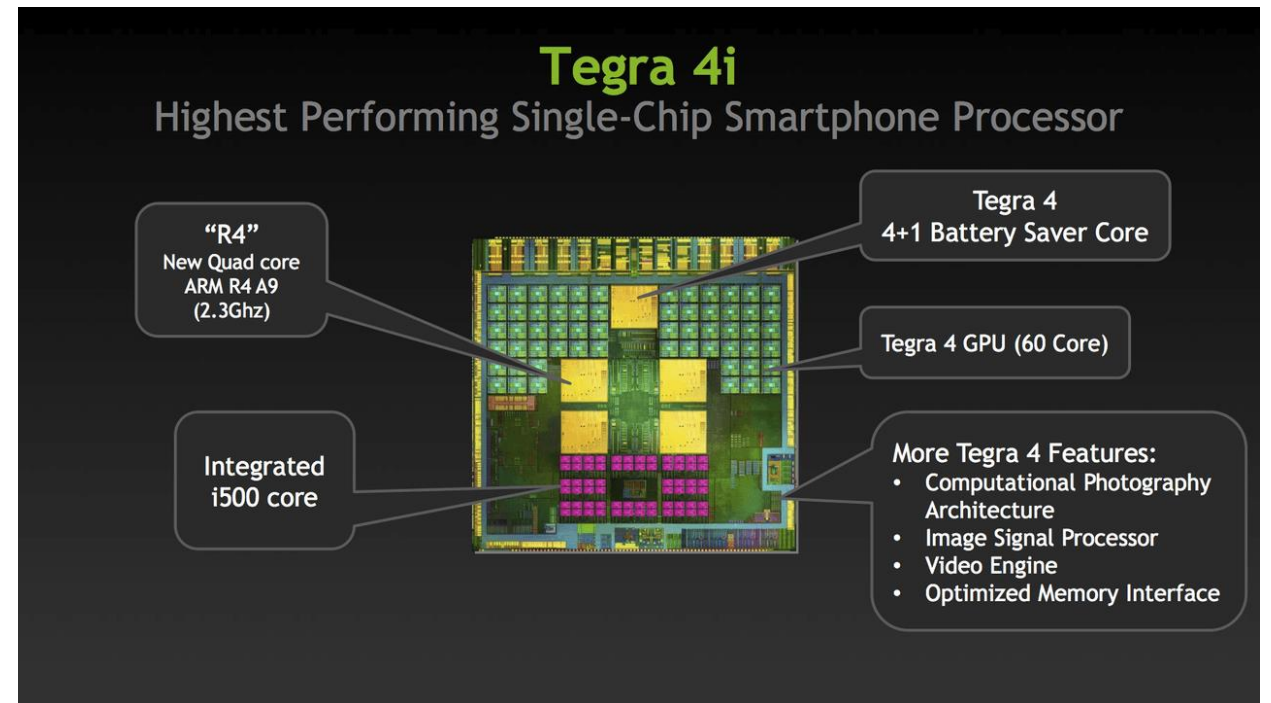


Figure 6.14: example of smartphone processor & GPU

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: GPU as data processors

- For many years, GPU architectures were designed very specifically to support the computational needs of **real-time three-dimensional scene rendering**.
- In recent years, users and GPU vendors have increasingly recognized that these devices are actually small-scale supercomputers suitable for use across a much broader range of applications.
- Modern GPUs provide floating-point execution speed measured in **trillions of floating-point operations per second (teraflops)**.
- *As of 2021, a high-end GPU configuration provides floating-point performance measured in the dozens of teraflops, and is capable of executing data-parallel mathematical algorithms hundreds of times faster than a standard desktop computer.*
- Taking advantage of the immense **parallel computing power** available in high-end GPUs, vendors of these devices have begun providing **programming interfaces** and expanded hardware capabilities to **enable the implementation of more generalized algorithms**.
- Of course, GPUs, even with enhancements to support general computing needs, are only truly effective at speeding up **algorithms** that exploit **data parallelization**.

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: GPU as data processors

- Some application areas that have proven to be suitable for GPU acceleration are:
 - **Big data:** In fields as diverse as *climate modeling*, *genetic mapping*, *business analytics*, and *seismic data analysis*, problem domains share the need to analyze enormous quantities of data, often measured in **terabytes** (TB) or **petabytes** (PB) (1 PB is 1,024 TB) in as efficient a manner as possible.
 - In many cases, these **analysis algorithms** iterate over **large datasets** searching for trends, correlations, and more sophisticated connections among what may seem at first to be disparate, unrelated masses of samples.
 - Until recently, analysis of these datasets at the appropriate level of granularity has often been perceived as infeasible due to the extensive execution time required for such an analysis.
 - *Today, however, many big data applications produce results in a reasonable length of time by combining the use of **GPU processing**, often on machines containing multiple interconnected GPU cards, and splitting the problem across multiple computer systems in a **cloud environment**.*
 - The use of **multiple computers**, each containing **multiple GPUs**, to execute **highly parallel algorithms** across an **enormous dataset** can be accomplished at a surprisingly low cost today in comparison to the historical costs of supercomputing systems.

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: Big data

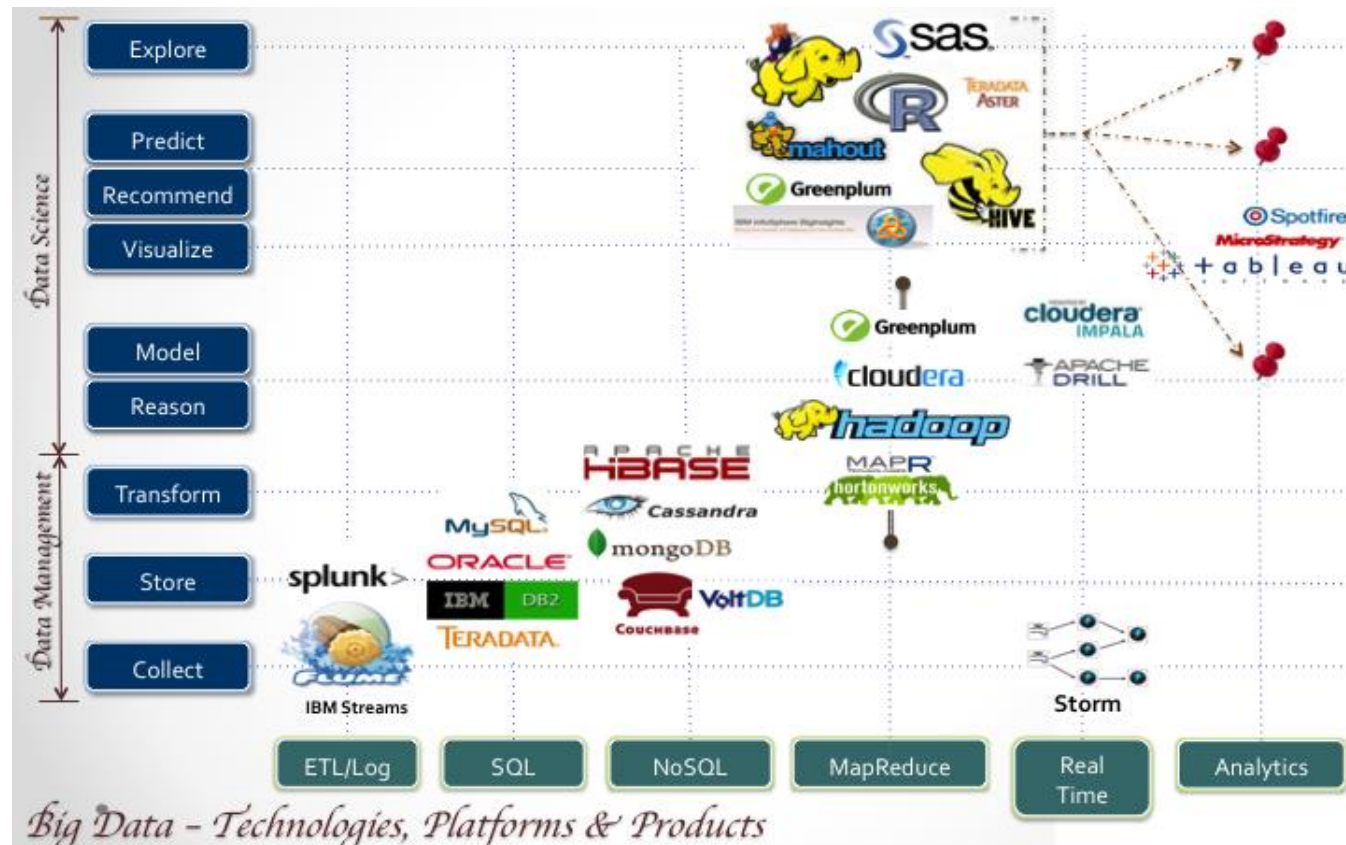


Figure 6.15: Big Data – Technologies, Platforms & Products

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: GPU as data processors

- Some application areas that have proven to be suitable for GPU acceleration are:
 - **Deep learning:**
 - **Deep learning** is a category of **artificial intelligence** (AI) that uses **multilayer networks** of **artificial neurons** to model the fundamental operations of the human brain.
 - A biological neuron is a type of nerve cell that processes information.
 - Neurons are interconnected via **synapses** and use electrochemical impulses to pass information to each other.
 - During **learning**, the human brain adjusts the connections among neurons to encode the information being learned for later retrieval. The human brain contains tens of billions of neurons.

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: GPU as data processors

- Some application areas that have proven to be suitable for GPU acceleration are:
 - **Artificial neural networks (ANNs)** use a software model of neuron behavior to mimic the learning and retrieval processes of the human brain.
 - Each artificial neuron receives input from potentially many other neurons and computes a single numeric output.
 - Some neurons are driven directly by the input data to be processed, and others produce outputs that are retrieved as the result of the ANN computation.
 - Each communication path between neurons has a **weighting** factor associated with it, which is simply a number that multiplies the strength of the signal traveling along that path.
 - The numeric input to a neuron is the **sum** of the **input signals** it receives, each multiplied by the **weight** of the associated path.
 - The neuron computes its output using a formula called the **activation function**.
 - The activation function determines the degree to which each neuron is "*triggered*" by its inputs.

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: GPU as data processors

- The following diagram represents an example of a single neuron that sums the inputs from three other neurons (N_1 - N_3), each multiplied by a weighting factor (w_1 - w_3).
- The **sum** passes to the **activation function**, $F(x)$, which produces the neuron's output signal.
- The use of three inputs in this example is arbitrary; in actual applications, each neuron can receive input from any number of other neurons (**Figure 6.16**).
- **ANNs** are organized in layers, where the first layer, called the **input layer**, is followed by one or more **internal layers** (called **hidden layers**), which are followed by an output layer.

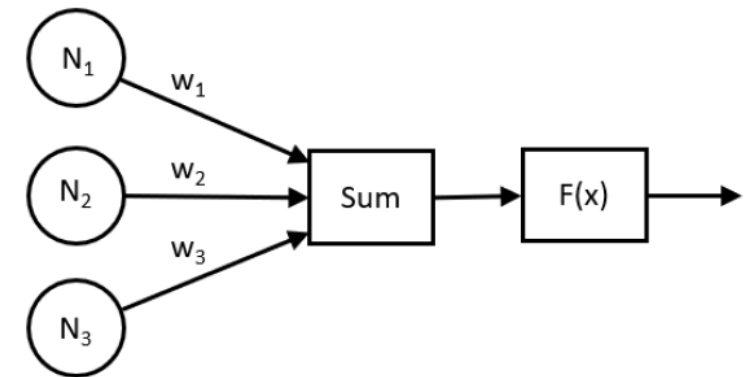


Figure 6.16: a neuron receiving inputs from three neurons

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: GPU as data processors

- Some ANN configurations are arranged in a data flow sequence from input to output, called a **feedforward network**, while other configurations provide **feedback** from some neurons to neurons in preceding layers.
- This configuration is called a **recurrent network**.
- The following screenshot shows an example of a simple feedforward network with three input neurons, a hidden layer consisting of four neurons, and two output neurons.
- This network is **fully connected**, meaning each neuron in the input and hidden layers connects to all neurons in the following layer.
- The connection weights are not shown in **Figure 6.17**.

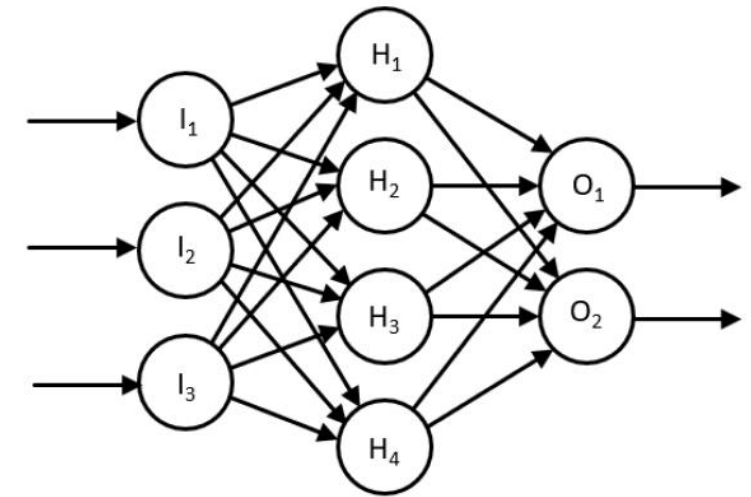


Figure 6.17: a three-layer feedforward network

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: Deep learning

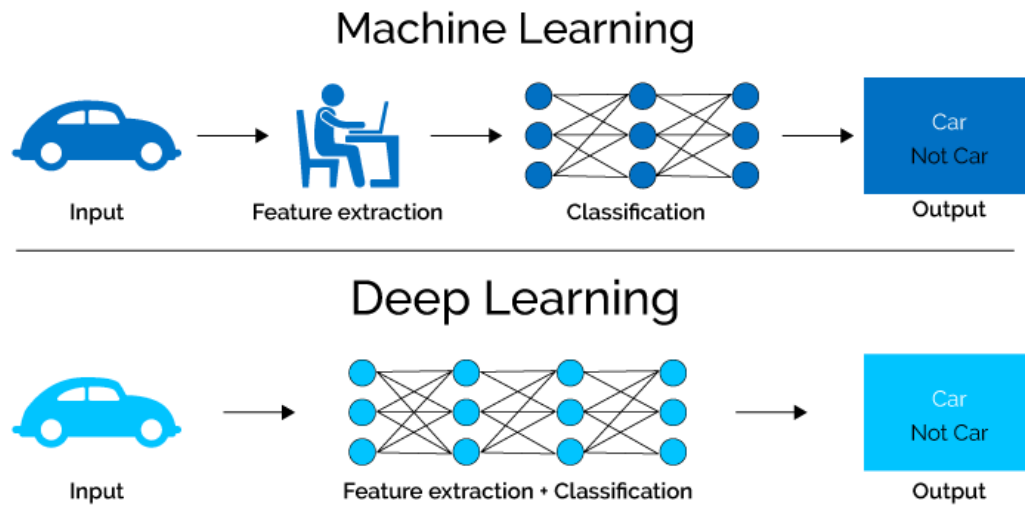


Figure 6.18: Machine learning versus Deep learning

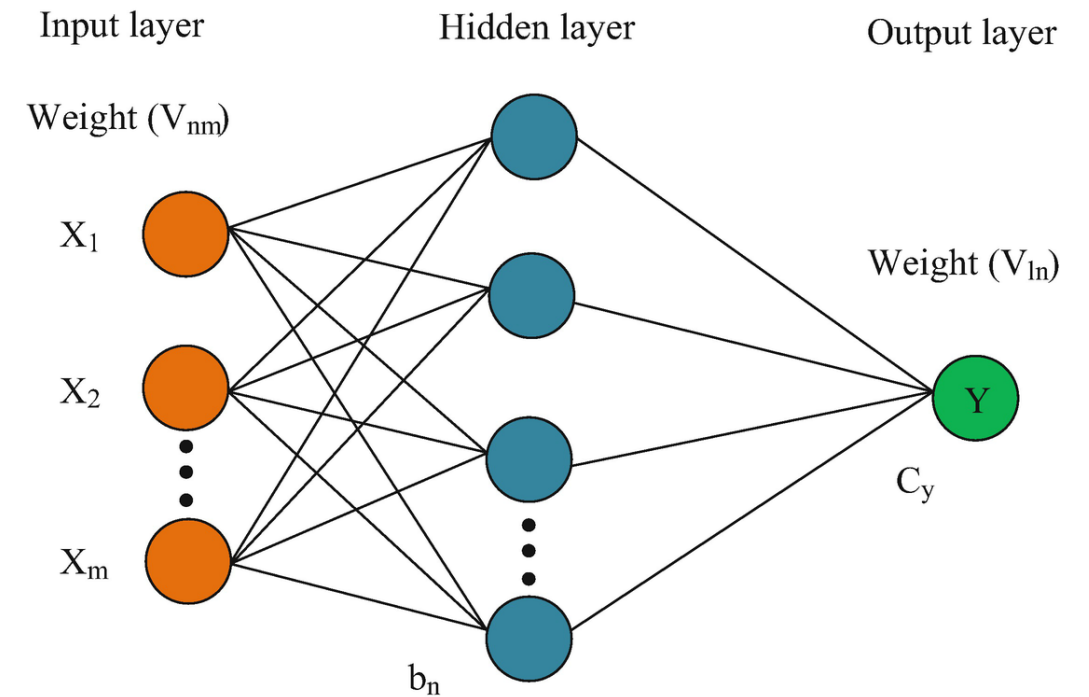


Figure 6.19: architecture of an Artificial Neural Network

— Computer Architecture

Chapter 6: Specialized Computing Domains

2. GPU processing: GPU as data processors

- Training an ANN consists of **adjusting** the **weighted connections** between neurons so that, when presented with a particular set of input data, the desired output is produced.
- Through the use of an **appropriate learning algorithm**, an ANN can be trained with a dataset composed of known correct outputs for a wide variety of inputs.
- Training a large, sophisticated ANN to perform a complex task such as driving a car or playing chess requires a **tremendous number of training iterations** drawn from a **very large dataset**. During training, each iteration makes small adjustments to the weighting factors within the network, slowly driving the network to a state of convergence.
- Once fully converged, the network is considered **trained** and can be used to produce outputs when presented with novel input. In other words, the network generalizes the information it learned during training and applies that knowledge to new situations.
- The feature that makes ANNs particularly suitable for **GPU processing** is their **parallel nature**.
- The human brain is effectively a **massively parallel computer** with billions of independent processing units. This form of parallelism is exploited during the ANN training phase to accelerate network convergence by performing the computations associated with multiple artificial neurons in parallel.

— **Computer Architecture**

Chapter 6: Specialized Computing Domains

Contents

1. Real-time computing
2. GPU processing
- 3. Examples of specialized architectures**

— Computer Architecture

Chapter 6: Specialized Computing Domains

3. Examples of specialized architectures

- This section examines some application-focused computing system configurations and highlights the specialized requirements addressed in the design of each.
- The configurations we will look at are as follows:
 - **Cloud compute server:**
 - A number of vendors offer access to **computing platforms** accessible to customers via the **Internet**. These **servers** allow users to load their own software applications onto the cloud server and perform any type of computation they desire.
 - *In general, these services bill their customers based on the type and quantity of computing resources being used and the length of time they are in use. The advantage for the customer is that these services cost nothing unless they are actually in use.*
 - At the higher end of performance, **servers** containing **multiple interconnected GPU cards** can be harnessed to perform **large-scale, floating-point intensive computations** on **huge datasets**.
 - In the **cloud** context, it is straightforward and often cost-effective to break a computation into smaller parts suitable for **parallel execution across multiple GPU-enabled servers**.
 - In this manner, it is feasible for organizations—and even individuals with limited funding—to harness computing capabilities that, until just a few years ago, were the exclusive province of government, big business, and research universities possessing the wherewithal to implement supercomputing facilities.

— Computer Architecture

Chapter 6: Specialized Computing Domains

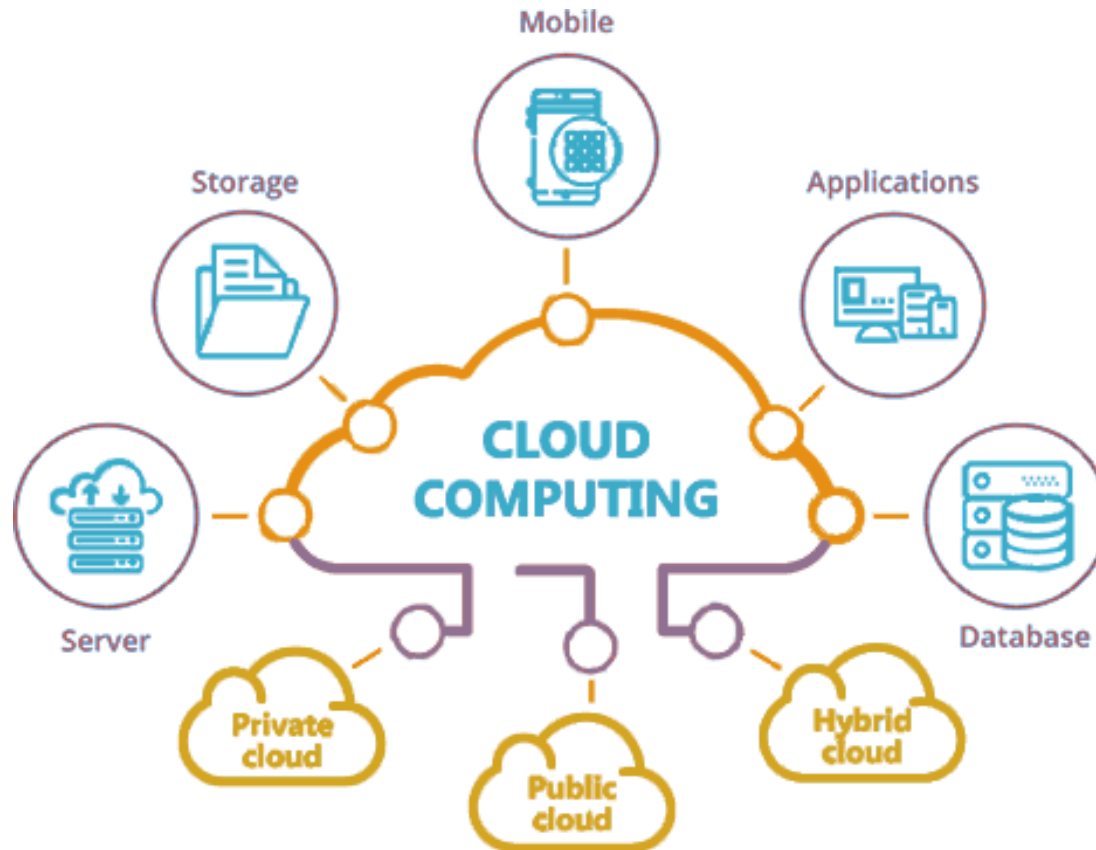


Figure 6.20: Cloud Computing

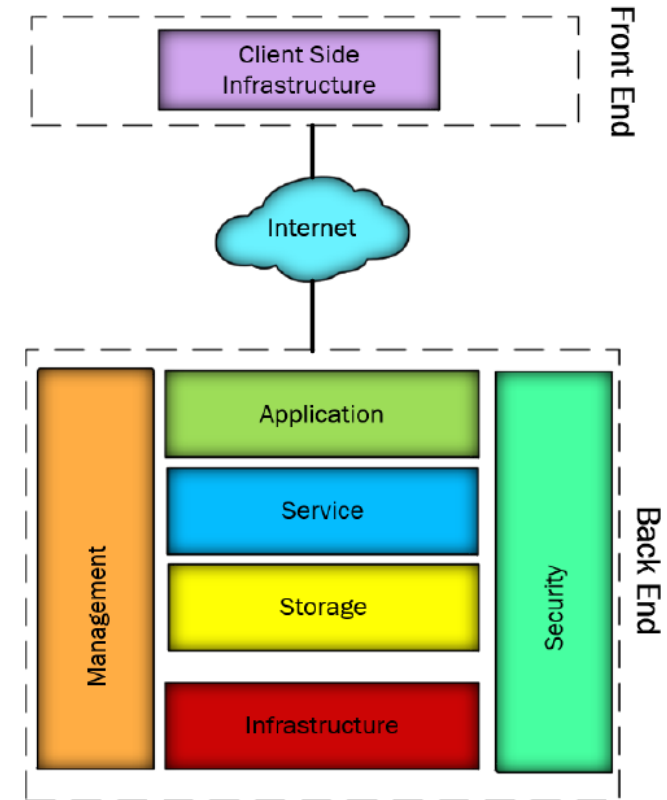


Figure 2: A generic Cloud Computing Architecture.

Figure 6.21: generic Cloud Computing Architecture

— Computer Architecture

Chapter 6: Specialized Computing Domains

3. Examples of specialized architectures

- The configurations we will look at are as follows:
 - **Business desktop computer:**
 - Business information technology managers strive to provide employees with the computing capability they need to do their jobs at the lowest cost.
 - Most office workers do not require exceptional graphics or computing performance, though their computer systems need to support modest video presentation requirements for such purposes as displaying employee training videos.
 - For **business** users, the **GPU integrated into modern processors** is usually **more than adequate**. For a reasonable price, business buyers can purchase computer systems containing processors in the midrange of performance with integrated graphics.
 - These systems provide full support for modern operating systems and standard office applications such as word processors, email, and spreadsheets. Should the need arise to expand a system's capabilities with higher-performance graphics, **the installation of a GPU in an expansion slot is a straightforward upgrade**.

Chapter 6: Specialized Computing Domains

3. Specialized architectures: GPU in business computers

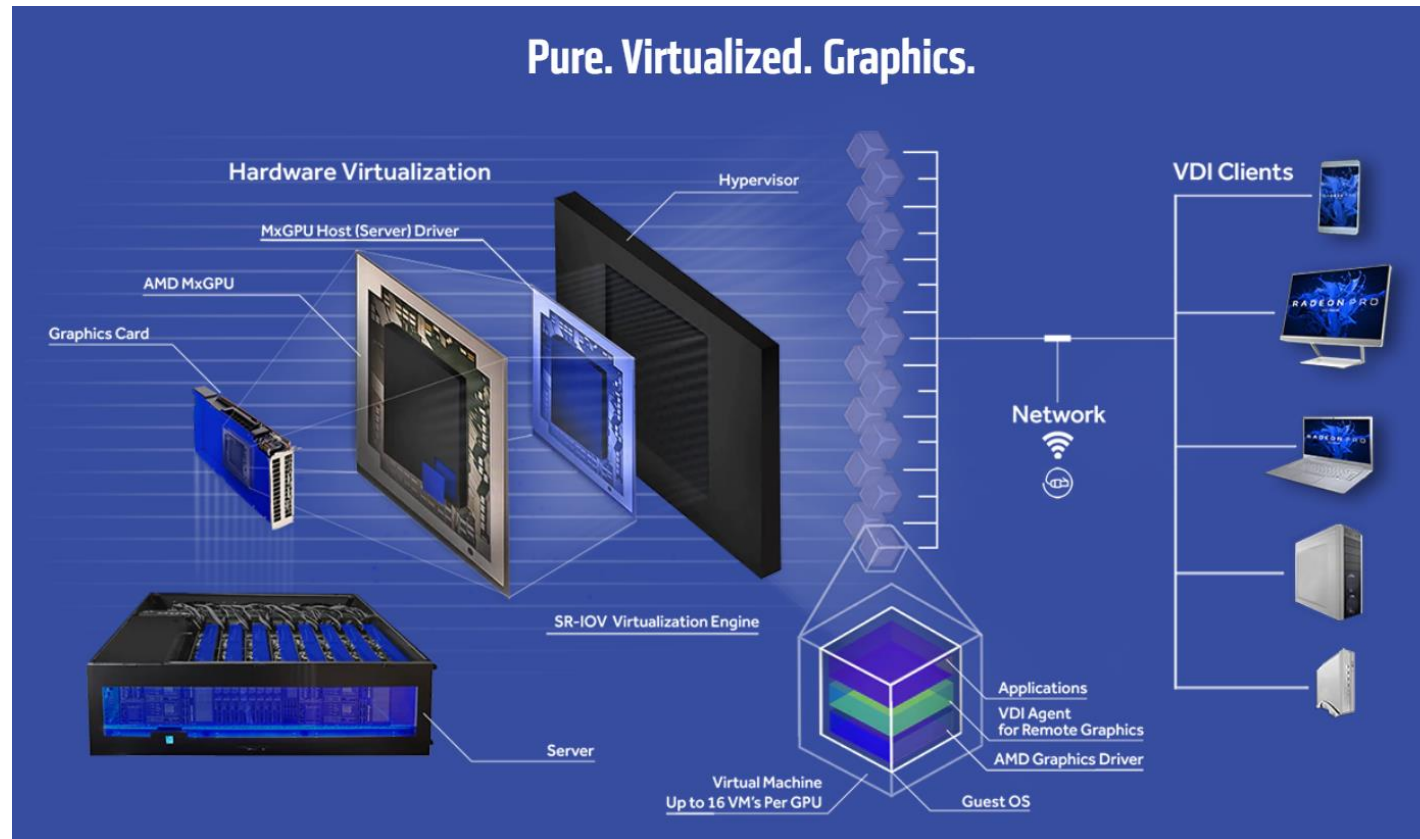


Figure 6.22: GPU virtualization

— Computer Architecture

Chapter 6: Specialized Computing Domains

3. Examples of specialized architectures

- The configurations we will look at are as follows:
 - **High-performance gaming computer:**
 - Computer gaming enthusiasts running the latest 3D games demand an **extreme level of GPU performance** to generate **detailed, high-resolution scenes** at the **highest achievable frame rate**.
 - These users are willing to invest in a powerful, power-hungry, and fairly costly **GPU** (*or even multiple GPUs*) to achieve the best possible graphics performance.
 - Almost as important as the **graphics performance**, a high-performance gaming computer must have a **fast system processor**.
 - The **processor** and **GPU** work together over the high-speed interface connecting them (typically **PCIe x16**) to determine the position and viewing direction of the scene observer, as well as the number, type, location, and orientation of all objects in the scene.
 - The **system processor** passes this geometric information to the **GPU**, which performs the mathematical operations necessary to render a lifelike image for display.
 - This process must repeat at a rate sufficient to deliver a smooth presentation of complex, rapidly changing scenes.

— Computer Architecture

Chapter 6: Specialized Computing Domains

3. Specialized architectures: GPU in business computers

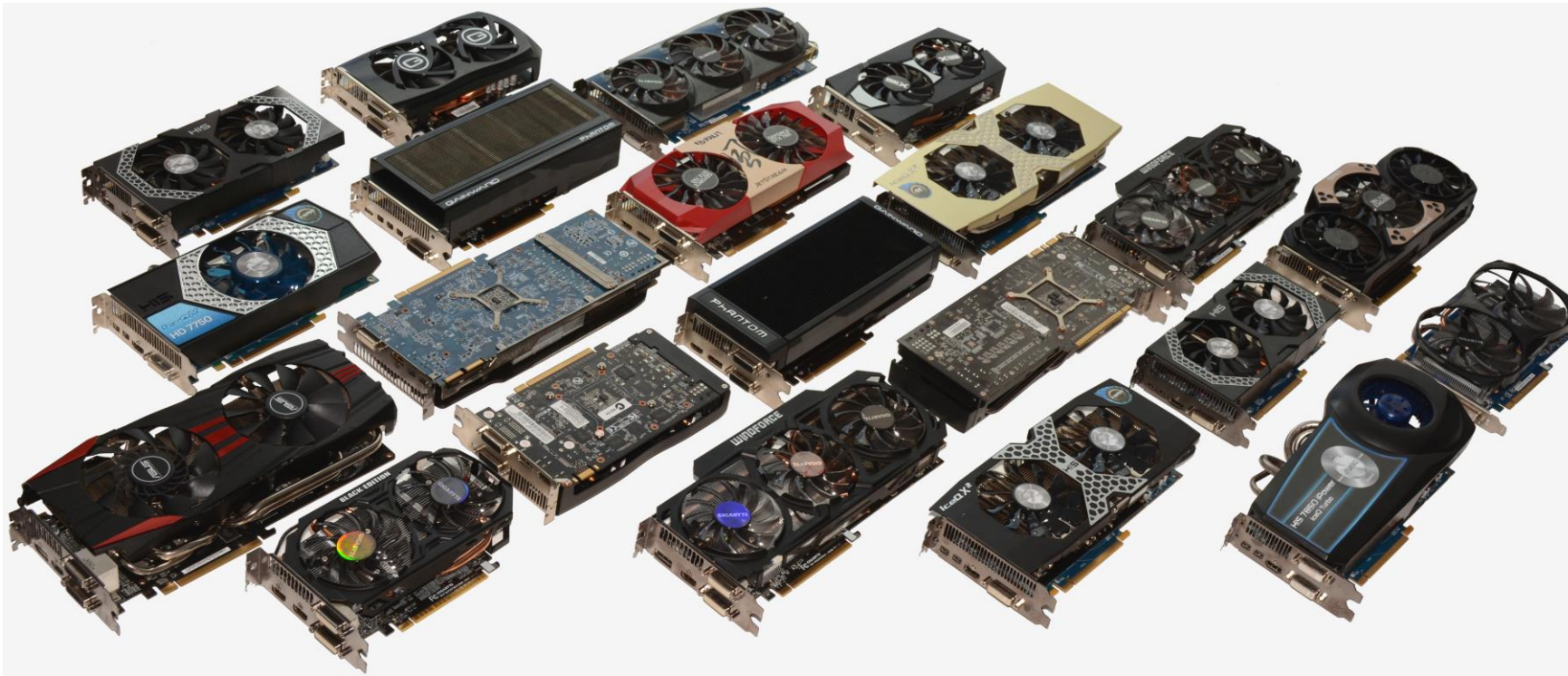


Figure 6.23: GPU virtualization

— Computer Architecture

Chapter 6: Specialized Computing Domains

3. Examples of specialized architectures

- The configurations we will look at are as follows:
 - **High-end smartphone:**
 - Today's smartphones combine **high-performance computational and graphic display capabilities** with **strict limits on power consumption and heat generation**. Users insist on fast, smooth, vibrant graphics for gaming and video display, but they will not tolerate this performance at the expense of unacceptably short battery life or a device that becomes hot to the touch.
 - Modern phone displays contain millions of full-color pixels, up to 12 GB of RAM, and support up to 1 TB of flash storage. These phones generally come with two **high resolution cameras** (one on the front and one on the back), capable of capturing still images and recording video.
 - High-end phones typically contain a **64-bit multi-core processor** with an **integrated GPU**, as well as a variety of features intended to achieve an optimal combination of energy efficiency and high performance.
 - **Smartphone architectures contain DSPs** to perform tasks such as **encoding** and **decoding voice audio** during telephone calls and processing the received and transmitted radio frequency signals flowing through the phone's various radio transceivers.
 - A typical phone includes support for **digital cellular service, Wi-Fi, Bluetooth**, and near-field communication (**NFC**). Modern smartphones are powerful, well-connected computing platforms optimized for operation under battery power.

— Computer Architecture

Chapter 6: Specialized Computing Domains

3. Specialized architectures: high-end smartphone

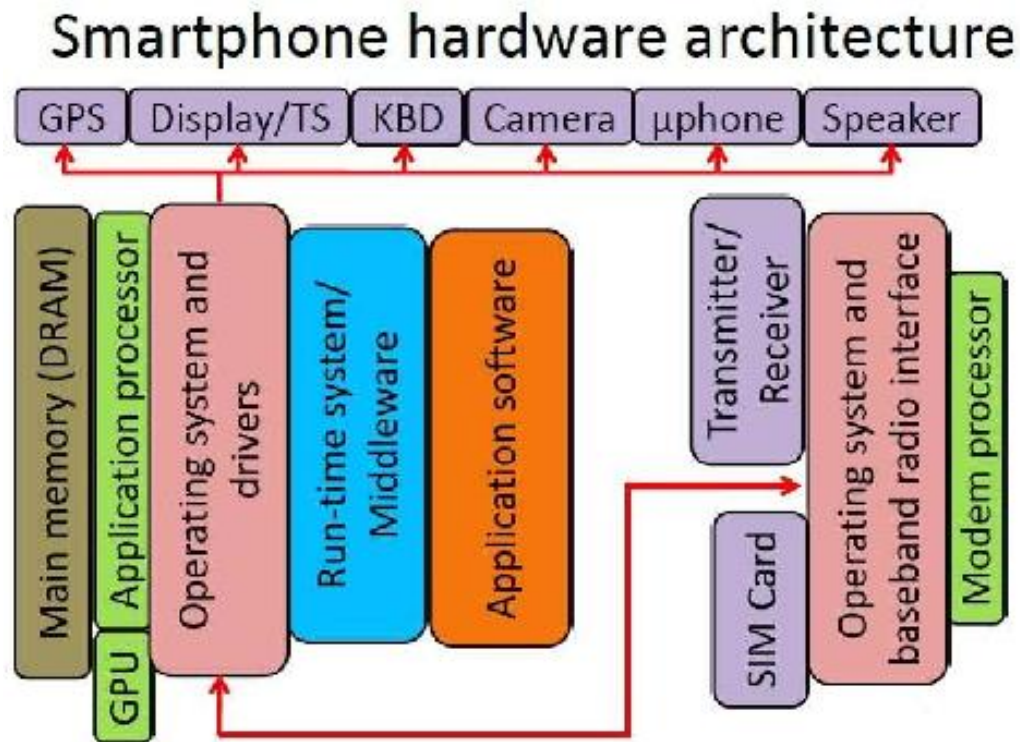


Figure 6.24: smartphone hardware architecture

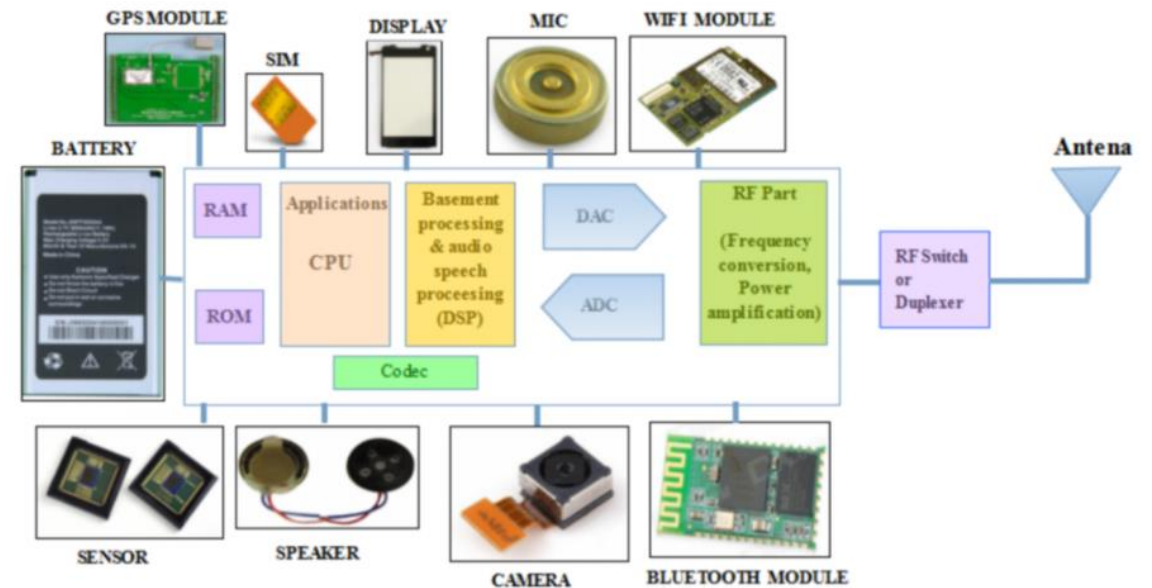
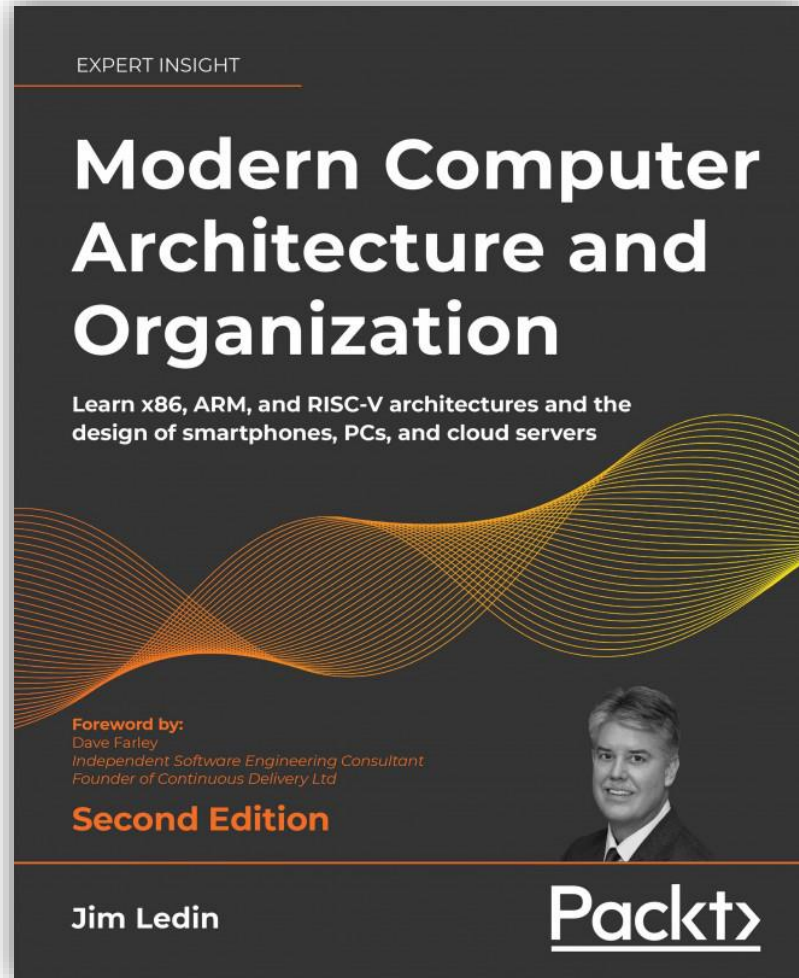


Figure 6.25: smartphone hardware architecture

Computer Architecture

End of Chapter 6 – Specialized Computing Domains



VIVES University of Applied Sciences
Bachelor in electronics-ICT

